Lists

CSCI 1101

2024-02-13

Lists

Conceptually, a string is just a finite sequence of characters.

Sometimes we want a finite sequence of other things.

For this we use **lists**.

We write lists with their elements enclosed in brackets and separated by commas:

```
[1, 2, 1, 2]              # a list of 4 ints
[True, False]             # a list of 2 bools
["Bow", "doin", "!"]      # a list of 3 strings
[]                        # a list of 0 ???
```

A list with nothing in it is called an **empty list**.

Homogeneous Lists

The purpose of a list is to contain its elements.

To do useful things with these elements we need to know their types.

So we usually work with lists whose elements all have the same type.
These are called **homogeneous lists**.

We will include the type of the list elements in the type specifications for homogeneous lists:

```
[1, 2, 1, 2]            #  list int
[True, False]           #  list bool
["Bow", "doin", "!"]    #  list str
[]                      #  list a
```

A **type variable** like “`a`” can stand for any type.

List Operators

The **concatenation** `_+_` and **repetition** `_*_` operators are overloaded to work with lists:

```
[1, 2] + [3, 4]
[True, False] * 3
```

Lists also support the `_in_` predicate, answering whether something is a **list element** (warning: *not* a sublist):

```
1 in [1, 2, 3]
[1, 2] in [1, 2, 3]
```

List Indexing and Slicing

Lists support the **index** `_[_]` and **slice** `_[_:_]` operators, just like strings:

```
[7, 11, 42][1]
["B", "o", "w", "d", "o", "i", "n"][1 : 5]
```

(This notation can be tricky to parse due to the overloading of brackets.)

Note that *list indexing* returns a *single list element*
while *list slicing* returns a *list of elements*.

For homogeneous lists these operators have the following types:

```
_[_]  :  (list a , int) --> a
_[_:_]  :  (list a , int , int) --> list a
```

List Functions

Python includes many built-in functions to work with lists.

Get the length of a list:

```
len  :  (list a) --> int
len([7, 8, 9])
```

Get the sorted version of a list:

```
sorted  :  (list a) --> list a
sorted([2, 1, 3])
```

Get the reversed version of a list:

```
reversed  :  (list a) --> iterator a
reversed([3, 2, 1])
```

Interlude: Iterators

It is a quirk of python that some functions that you expect to return a list return an iterator instead.

An **iterator** is something that can produce a list on demand.

We can get a list from an iterator using the `list` type-conversion function:

```
list(reversed([3, 2, 1]))
```

Some functions that want a `list` argument can accept an `iterator` instead.

Activity: List Functions

Write a function `reverse_sorted : (list a) --> list a`
that takes a list of elements of some type and returns the list containing these elements in the *opposite* of the normal sorting order.

For example:

```
> reverse_sorted([2, 3, 1])
  [3, 2, 1]
> reverse_sorted(["aardvark", "monkey", "Zebra"])
  ['monkey', 'aardvark', 'Zebra']
```

Working With Lists of Strings

The `list` type-conversion function is also good for “exploding” a string into the list of its characters:

```
list("Bowdoin")
```

To separate a string into a list of words use the function:

```
str.split  :  (str) --> list str
str.split("it was the best of times")
```

To put them back together again use the function:

```
str.join  :  (str , list str)  --> str
str.join("_" , ["was", "it", "really", "?"])
```

The first argument is a “glue” string that gets inserted between the list elements.

Activity: Reversing a String

Use the functions `reversed`, `list`, and `str.join` to write a function `rev_string : (str) --> str` that reverses the order of the characters in a string:

For example:

```
> rev_string("Bowdoin")
  'niodwoB'
```

More List Functions

More list-related functions live in the `list` module.

Get the number of occurrences of a list element:

```
list.count  :  (list a , a) --> int
list.count([7, 8, 8, 9] , 8)
list.count([7, 8, 8, 9] , 6)
```

Get the index of the first occurrence of a list element (it's an error if not present):

```
list.index  :  (list a , a) --> int
list.index([7, 8, 8, 9] , 8)
list.index([7, 8, 8, 9] , 6)
```

There are also `count` and `index` function for strings:

```
str.count("Bowdoin" , "o")
str.index("Bowdoin" , "oi")
```

String Interpolation

A topic that I should have covered when we studies strings is **string interpolation**.

This is when we concatenate a string with the result of implicitly calling the `str` function on an expression.

In python we do this by putting an `f` or `F` *before* the opening quote of a string.

We place expressions that we want to interpolate between braces `{` and `}`.

For example:

```
f"did you know 2 + 2 = {2 + 2}?"
```

is the same as:

```
"did you know 2 + 2 = " + str(2 + 2) + "?"
```

Heterogeneous Equalities

When we were discussing the equality predicate `_==_`
I should have stressed that although there are a few special cases
where equalities between elements of different types are true:

```
3 == 3.0      # kinda makes sense, maybe
1 == True     # pretty dubious, I'd say
```

Usually, such comparisons between elements of different types are *false*:

```
True != "True"       # because bool != str
42 != "42"           # because int != str
```

Except for equalities between elements of different numerical types,
you don't need to remember the exceptions and can assume that
if `type(x) != type(y)` then `x != y`.

To Do

Finish *Homework 3: Conditionals* on CodeRunner by 11:00AM this Thursday.

Study for Midterm 1 in class next Tuesday, February 20
(practice tests available on Canvas and CodeRunner).

One index card or 1/3 page paper of notes allowed.

Bring your questions to the review session this Friday.

Finish on *Project 1: The Turtle Module* on CodeRunner by next Friday.